1. Question and approach

2. Setup and data loading

3. Cleaning

4. Exploratory look

5. Train, tune, and test split

6. Scoring function

7. Baselines

8. Model 1: multinomial logistic regression

9. Model 2: gradient boosting

10. Results on the held-out test seasons (2024-2025)

11. Drift: has coaching behavior changed?

12. Where is the gap largest?

13. Calibration of the “go” probability (test seasons)

14. Interpretation, limitations, next steps, contributions

AI Statement

Predicting NFL Fourth Downs: Milestone Report

Chaikou Diallo and Dillon Mavunga

1. Question and approach

Research question: How accurately can we predict a coach’s fourth-down choice (go for it, punt, or field goal) from the game situation, and does a model trained on older seasons still predict recent seasons well?

Why this approach fits. The coach’s choice is a three-class outcome, so we treat it as a multiclass classification problem. We compare two models against simple baselines:

- **Multinomial logistic regression** with spline terms. It is simple, easy to read, and a strong check on whether a fancy model is even needed.
- **Gradient boosting (xgboost).** Fourth-down choices depend on interactions (for example, 4th-and-2 means something very different at midfield than at your own 20), and trees pick up those interactions without us writing them by hand.

Why we split by season. Coaching behavior changes over time, so a random split would let the model learn from the future. We train on 1999 to 2021, use 2022 to 2023 to pick model settings, and keep 2024 to 2025 locked away as the final test.

The drift idea. If coaches have become more aggressive, a model trained on older seasons should *under-predict* going for it in recent seasons. So the gap between predicted and actual go-for-it rates is a direct, measurable test of behavior change.

2. Setup and data loading

Data comes from the nflverse play-by-play files (https://github.com/nflverse/nflfastR (https://github.com/nflverse/nflfastR)), loaded with `nflreadr`. Anyone can rerun this notebook; it downloads the data on its own.

```
#install.packages(c("nflreadr", "tidyverse", "tidymodels", "xgboost", "nnet", "knitr", "nfl4th"))
library(nflreadr)
library(tidyverse)
library(tidymodels)
library(splines)
library(nnet)
library(knitr)
library(nfl4th)
```

```
#install.packages("knitr")
```

```
library(knitr)
```

Loading every season at once (over a million rows, about 370 columns) can use several GB of memory. To keep RStudio fast, we load one season at a time, keep only fourth downs, and keep only the columns we need.

```
seasons = 1999:2025

keep_cols = c("season", "season_type", "game_id", "posteam", "down", "ydstogo",
              "yardline_100", "score_differential", "game_seconds_remaining",
              "half_seconds_remaining", "qtr", "posteam_timeouts_remaining",
              "defteam_timeouts_remaining", "posteam_type", "spread_line", "play_type")

FourDown_Full = map_dfr(seasons, function(s) {
  load_pbp(s) |>
    subset(down == 4) |>
    select(all_of(keep_cols)) |>
    as_tibble()
})

count(FourDown_Full, play_type, sort = TRUE)
```

play_type	n
punt	66137

play_type	n
field_goal	25455
pass	9629
run	5885
no_play	5727
NA	218
qb_kneel	83
	22

3. Cleaning

Choices we made, and why:

- **Regular season only.** Playoff games are few and may follow different logic.
- **Keep only real decisions:** `punt`, `field_goal`, `run`, `pass`. We drop `no_play` (mostly penalties, where the play didn't count) and `qb_kneel`.
- **Run and pass become "go".** Our target has three classes: `go`, `punt`, `field_goal`.
- **Spread from the offense's view.** `spread_line` is from the home team's view, so we flip it for away teams. Positive means the offense was favored.
- **Only pre-snap features.** Columns like `epa`, `wpa`, and `fourth_down_converted` are recorded after the play, so using them would leak the answer.

```
features = c("ydstogo", "yardline_100", "score_differential", "game_seconds_remaini
ng",
             "half_seconds_remaining", "qtr", "posteam_timeouts_remaining",
             "defteam_timeouts_remaining", "home", "spread_pos")

FourDown_Clean = FourDown_Full |>
  subset(season_type == "REG" & play_type %in% c("punt", "field_goal", "run", "pas
s")) |>
  mutate(
    choice     = if_else(play_type %in% c("run", "pass"), "go", play_type),
    choice     = factor(choice, levels = c("field_goal", "go", "punt")),
    home       = as.integer(posteam_type == "home"),
    spread_pos = if_else(home == 1, spread_line, -spread_line)
  ) |>
  drop_na(all_of(features))

nrow(FourDown_Clean)
```

```
## [1] 102661
```

4. Exploratory look

```
FourDown_Clean |>
  mutate(era = case_when(season <= 2017 ~ "1999–2017",
                         season <= 2021 ~ "2018–2021",
                         TRUE           ~ "2022–2025")) |>
  count(era, choice) |>
  group_by(era) |>
  mutate(share = round(n / sum(n), 3)) |>
  select(-n) |>
  pivot_wider(names_from = choice, values_from = share) |>
  kable(caption = "Share of fourth downs by choice and era")
```

Share of fourth downs by choice and era

era	field_goal	go	punt
1999-2017	0.233	0.124	0.643
2018-2021	0.246	0.178	0.576
2022-2025	0.254	0.204	0.542

Punts are the most common choice in every era, so the classes are imbalanced. Going for it rose from about 12% to about 20%.

5. Train, tune, and test split

```
train = subset(FourDown_Clean, season <= 2021)
tune  = subset(FourDown_Clean, season %in% 2022:2023)
test  = subset(FourDown_Clean, season >= 2024)

tibble(set = c("train (1999–2021)", "tune (2022–2023)", "test (2024–2025)"),
       plays = c(nrow(train), nrow(tune), nrow(test))) |>
  kable()
```

set	plays
train (1999-2021)	87089
tune (2022-2023)	7953
test (2024-2025)	7619

6. Scoring function

We score with **log loss** (main metric; punishes confident wrong guesses), **Brier score** (computed step by step below), **accuracy**, **macro F1** (treats all three classes equally, so the model can't win by always guessing punt), and **go recall** (of the times teams actually went for it, how often the model called it).

```
class_levels = c("field_goal", "go", "punt")

score_model = function(df, model_name) {
  # df needs: choice, .pred_field_goal, .pred_go, .pred_punt
  P = as.matrix(df[, c(".pred_field_goal", ".pred_go", ".pred_punt")])
  Y = model.matrix(~ choice - 1, data = df)          # one-hot of the true class
  pred_class = factor(class_levels[max.col(P, ties.method = "first")], levels = cla
ss_levels)
  scored = mutate(df, .pred_class = pred_class)

  tibble(
    model     = model_name,
    log_loss  = mn_log_loss(scored, choice, .pred_field_goal, .pred_go, .pred_punt)
$.estimate,
    brier     = mean(rowSums((P - Y)^2)),              # manual multiclass Brier
    accuracy  = mean(pred_class == df$choice),
    macro_f1  = f_meas(scored, choice, .pred_class, estimator = "macro")$.estimate,
    go_recall = mean(pred_class[df$choice == "go"] == "go")
  )
}
```

7. Baselines

Class-share baseline: predicts the training share of each class for every play (so it always picks punt). Any real model has to beat this.

Lookup-table baseline: buckets each play by field position and yards to go, then predicts the training mix of choices in that bucket. This is roughly how a fan would think about it (“4th-and-1 at midfield, teams usually go”).

```
class_share = prop.table(table(train$choice))

baseline_share = test |>
  mutate(.pred_field_goal = class_share[["field_goal"]],
         .pred_go         = class_share[["go"]],
         .pred_punt       = class_share[["punt"]])

add_buckets = function(df) {
  df |>
    mutate(fp_bin  = cut(yardline_100, c(0, 10, 20, 30, 40, 50, 60, 70, 80, 100)),
           ytg_bin = cut(pmin(ydstogo, 11), c(0, 1, 2, 3, 5, 7, 10, 99)))
}

lookup = train |>
  add_buckets() |>
  count(fp_bin, ytg_bin, choice, .drop = FALSE) |>
  group_by(fp_bin, ytg_bin) |>
  mutate(prob = (n + 1) / sum(n + 1)) |>              # +1 smoothing for empty buc
kets
  ungroup() |>
  select(-n) |>
  pivot_wider(names_from = choice, values_from = prob, names_prefix = ".pred_")

baseline_lookup = test |>
  add_buckets() |>
  left_join(lookup, by = c("fp_bin", "ytg_bin"))
```

```
#install.packages("GPfit")
```

```
library(tidymodels)
```

8. Model 1: multinomial logistic regression

We put natural splines on the four main numeric features so the model can bend (for example, the effect of yards to go is not a straight line). We tune the spline flexibility (degrees of freedom) on 2022 to 2023.

```
make_logit_formula = function(k) {
  as.formula(paste0(
    "choice ~ ns(ydstogo, ", k, ") + ns(yardline_100, ", k, ") + ",
    "ns(score_differential, ", k, ") + ns(game_seconds_remaining, ", k, ") + ",
    "half_seconds_remaining + qtr + posteam_timeouts_remaining + ",
    "defteam_timeouts_remaining + home + spread_pos"))
}

predict_logit = function(fit, df) {
  probs = predict(fit, newdata = df, type = "probs")
  df |> mutate(.pred_field_goal = probs[, "field_goal"],
               .pred_go         = probs[, "go"],
               .pred_punt       = probs[, "punt"])
}

logit_tuning = map_dfr(c(3, 5, 8), function(k) {
  fit = multinom(make_logit_formula(k), data = train, maxit = 1000, trace = FALSE)
  score_model(predict_logit(fit, tune), paste("logit df =", k))
})
logit_tuning |> select(model, log_loss) |> kable(digits = 3, caption = "Logistic tu
ning (2022–2023)")
```

Logistic tuning (2022-2023)

model	log_loss
logit df = 3	0.337
logit df = 5	0.334
logit df = 8	0.332

```
best_k = c(3, 5, 8)[which.min(logit_tuning$log_loss)]
logit_fit = multinom(make_logit_formula(best_k), data = train, maxit = 1000, trace
= FALSE)
```

9. Model 2: gradient boosting

We tune two settings on 2022 to 2023: learning rate and tree depth. We keep the grid small on purpose so we don’t overfit the tuning seasons.

```
model_data = function(df) select(df, choice, all_of(features))

fit_gbm = function(lr, depth) {
  boost_tree(trees = 400, learn_rate = lr, tree_depth = depth) |>
    set_engine("xgboost") |>
    set_mode("classification") |>
    fit(choice ~ ., data = model_data(train))
}

predict_gbm = function(fit, df) bind_cols(df, predict(fit, model_data(df), type =
"prob"))

gbm_grid = crossing(lr = c(0.05, 0.1), depth = c(3, 6))

set.seed(2026)
gbm_tuning = pmap_dfr(gbm_grid, function(lr, depth) {
  fit = fit_gbm(lr, depth)
  score_model(predict_gbm(fit, tune), paste0("gbm lr = ", lr, ", depth = ", depth))
|>
    mutate(lr = lr, depth = depth)
})
gbm_tuning |> select(model, log_loss) |> kable(digits = 3, caption = "Boosting tuni
ng (2022–2023)")
```

Boosting tuning (2022-2023)

model	log_loss
gbm lr = 0.05, depth = 3	0.282
gbm lr = 0.05, depth = 6	0.278
gbm lr = 0.1, depth = 3	0.277
gbm lr = 0.1, depth = 6	0.284

```
best_gbm = gbm_tuning |> slice_min(log_loss, n = 1)
gbm_fit  = fit_gbm(best_gbm$lr, best_gbm$depth)
```

Note: after picking settings, we keep the models trained on 1999 to 2021 only. That keeps the test seasons fully untouched and keeps the drift test clean.

10. Results on the held-out test seasons (2024-2025)

```
results = bind_rows(
  score_model(baseline_share,                 "Class-share baseline"),
  score_model(baseline_lookup,                "Lookup-table baseline"),
  score_model(predict_logit(logit_fit, test), "Multinomial logistic"),
  score_model(predict_gbm(gbm_fit, test),     "Gradient boosting")
)
kable(results, digits = 3, caption = "Test performance, 2024–2025 (lower log loss a
nd Brier are better)")
```

Test performance, 2024-2025 (lower log loss and Brier are better)

model	log_loss	brier	accuracy	macro_f1	go_recall
Class-share baseline	1.054	0.632	0.522	0.686	0.000
Lookup-table baseline	0.477	0.287	0.794	0.714	0.331
Multinomial logistic	0.389	0.235	0.833	0.770	0.423
Gradient boosting	0.341	0.202	0.866	0.823	0.540

```
test_gbm = predict_gbm(gbm_fit, test) |>
  mutate(.pred_class = factor(class_levels[max.col(
    cbind(.pred_field_goal, .pred_go, .pred_punt), ties.method = "first")], levels
= class_levels))

conf_mat(test_gbm, truth = choice, estimate = .pred_class)
```

```
##            Truth
## Prediction   field_goal   go punt
##   field_goal       1761  276    2
##   go                 77  889   27
##   punt              156  481 3950
```

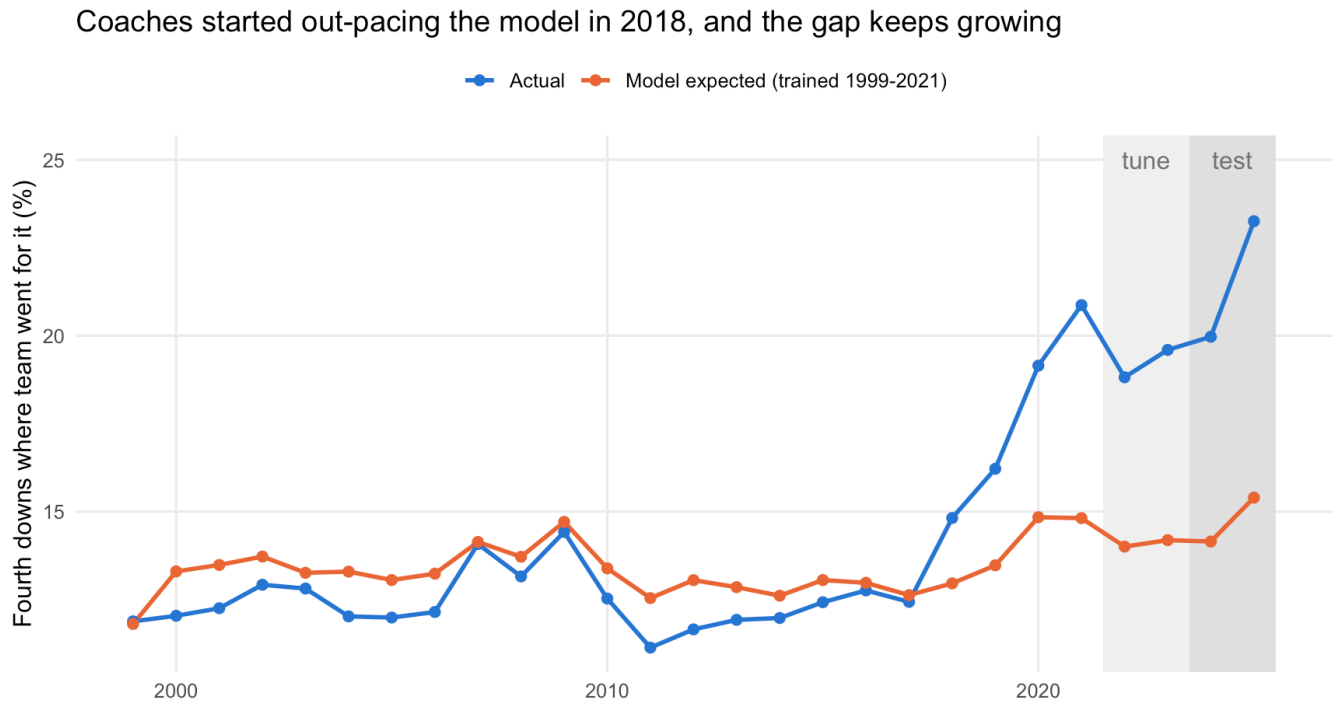
11. Drift: has coaching behavior changed?

We use the boosting model (trained on 1999-2021) to predict every season, then compare the model's expected go-for-it rate to the real rate.

```
all_preds = predict_gbm(gbm_fit, FourDown_Clean)

drift = all_preds |>
  group_by(season) |>
  summarise(Actual = mean(choice == "go"),
            `Model expected (trained 1999-2021)` = mean(.pred_go),
            .groups = "drop")

drift |>
  pivot_longer(-season, names_to = "series", values_to = "rate") |>
  ggplot(aes(season, rate * 100, color = series)) +
  annotate("rect", xmin = 2021.5, xmax = 2023.5, ymin = -Inf, ymax = Inf, fill = "g
rey95") +
  annotate("rect", xmin = 2023.5, xmax = 2025.5, ymin = -Inf, ymax = Inf, fill = "g
rey88") +
  annotate("text", x = c(2022.5, 2024.5), y = 25, label = c("tune", "test"), color
= "grey45") +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_color_manual(values = c("Actual" = "#2a78d6",
                                "Model expected (trained 1999-2021)" = "#eb6834"))
+
  labs(title = "Coaches started out-pacing the model in 2018, and the gap keeps gro
wing",
       x = NULL, y = "Fourth downs where team went for it (%)", color = NULL) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top", panel.grid.minor = element_blank())
```



```
all_preds |>
  subset(season >= 2014) |>
  group_by(season) |>
  mn_log_loss(choice, .pred_field_goal, .pred_go, .pred_punt) |>
  select(season, log_loss = .estimate) |>
  kable(digits = 3, caption = "Model log loss by season (it gets worse as behavior
drifts)")
```

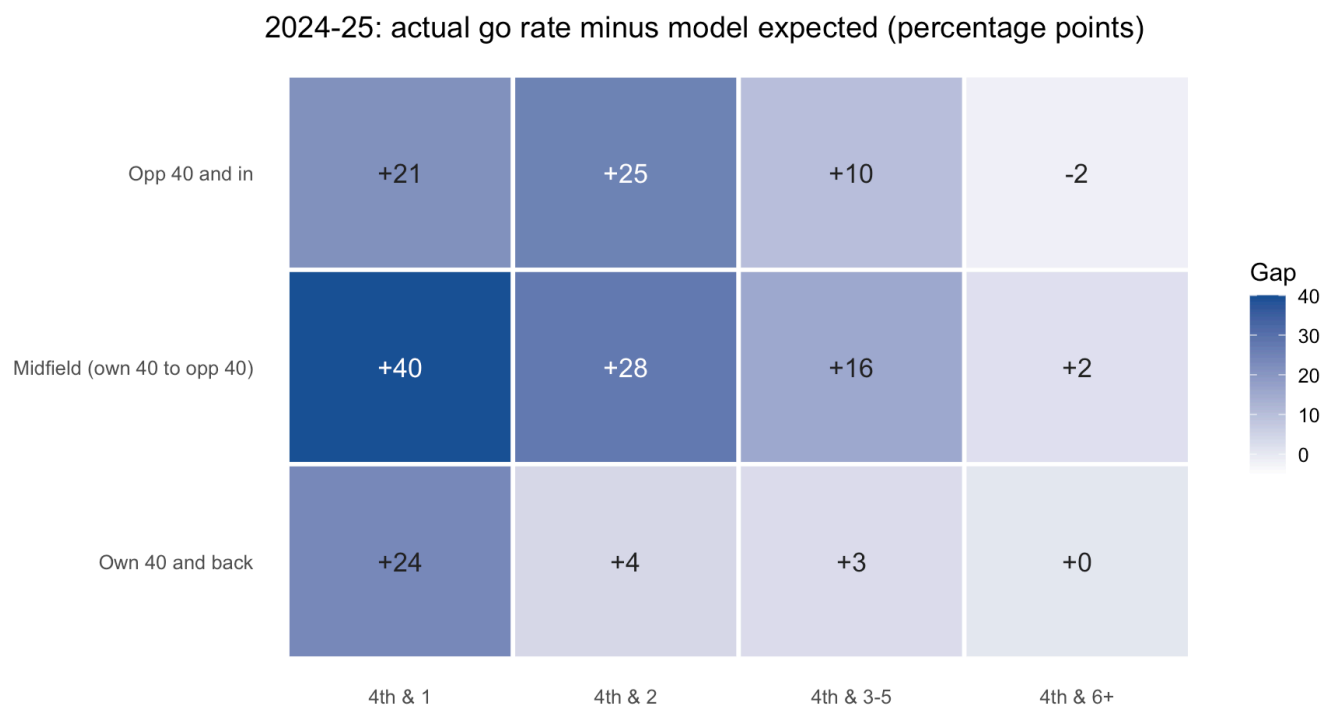
Model log loss by season (it gets worse as behavior drifts)

season	log_loss
2014	0.158
2015	0.157
2016	0.169
2017	0.170
2018	0.188
2019	0.210
2020	0.242
2021	0.274
2022	0.268
2023	0.286
2024	0.321
2025	0.360

12. Where is the gap largest?

```
gap = all_preds |>
  subset(season >= 2024) |>
  mutate(
    field_pos = cut(yardline_100, c(0, 40, 60, 100),
                    labels = c("Opp 40 and in", "Midfield (own 40 to opp 40)", "Own
40 and back")),
    distance  = cut(ydstogo, c(0, 1, 2, 5, 99), labels = c("4th & 1", "4th & 2", "4
th & 3-5", "4th & 6+"))
  ) |>
  group_by(field_pos, distance) |>
  summarise(plays = n(), actual = mean(choice == "go"), expected = mean(.pred_go),
.groups = "drop") |>
  mutate(gap_pts = 100 * (actual - expected))

ggplot(gap, aes(distance, fct_rev(field_pos), fill = gap_pts)) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = sprintf("%+.0f", gap_pts), color = gap_pts > 25), size = 4.
5) +
  scale_fill_gradient(low = "#ffffff", high = "#184f95", limits = c(-5, 40)) +
  scale_color_manual(values = c(`TRUE` = "white", `FALSE` = "grey15"), guide = "non
e") +
  labs(title = "2024-25: actual go rate minus model expected (percentage points)",
       x = NULL, y = NULL, fill = "Gap") +
  theme_minimal(base_size = 12) +
  theme(panel.grid = element_blank())
```




```
kable(gap, digits = 3)
```

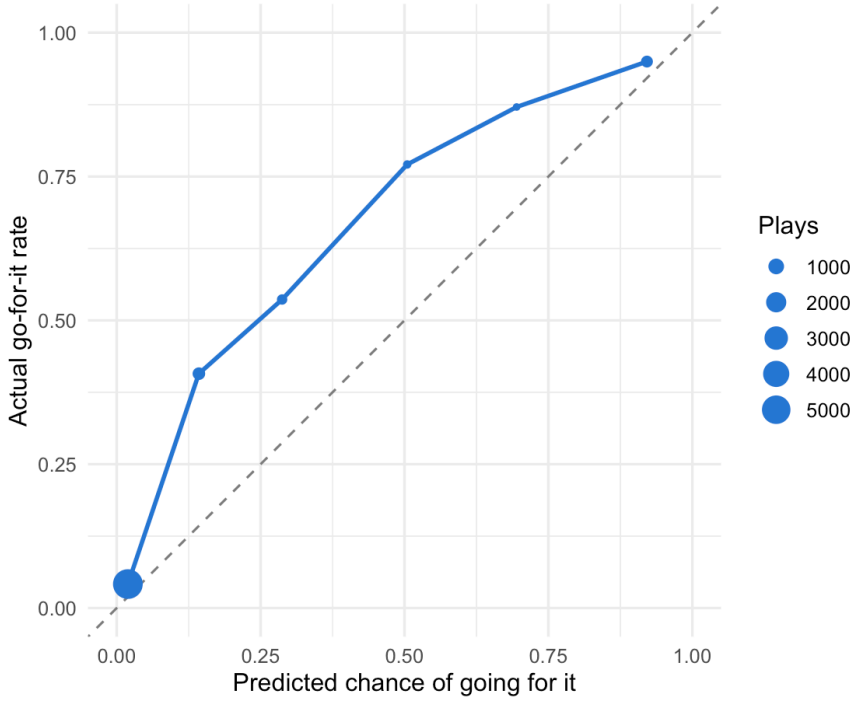
field_pos	distance	plays	actual	expected	gap_pts
Opp 40 and in	4th & 1	398	0.894	0.680	21.456

field_pos	distance	plays	actual	expected	gap_pts
Opp 40 and in	4th & 2	267	0.584	0.331	25.320
Opp 40 and in	4th & 3-5	804	0.301	0.202	9.932
Opp 40 and in	4th & 6+	1443	0.093	0.108	-1.523
Midfield (own 40 to opp 40)	4th & 1	239	0.858	0.460	39.768
Midfield (own 40 to opp 40)	4th & 2	128	0.508	0.226	28.169
Midfield (own 40 to opp 40)	4th & 3-5	440	0.311	0.153	15.842
Midfield (own 40 to opp 40)	4th & 6+	1105	0.105	0.087	1.797
Own 40 and back	4th & 1	258	0.376	0.140	23.570
Own 40 and back	4th & 2	167	0.108	0.070	3.734
Own 40 and back	4th & 3-5	519	0.081	0.054	2.650
Own 40 and back	4th & 6+	1851	0.042	0.038	0.455

13. Calibration of the “go” probability (test seasons)

If the model says 30%, teams should go about 30% of the time. Points above the diagonal mean coaches went for it more than the model expected.

```
all_preds |>
  subset(season >= 2024) |>
  mutate(bin = cut(.pred_go, c(0, 0.1, 0.2, 0.4, 0.6, 0.8, 1))) |>
  group_by(bin) |>
  summarise(plays = n(), predicted = mean(.pred_go), actual = mean(choice == "go"),
.groups = "drop") |>
  ggplot(aes(predicted, actual)) +
  geom_abline(linetype = "dashed", color = "grey50") +
  geom_line(color = "#2a78d6", linewidth = 1) +
  geom_point(aes(size = plays), color = "#2a78d6") +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(x = "Predicted chance of going for it", y = "Actual go-for-it rate", size =
"Plays") +
  theme_minimal(base_size = 12)
```



14. Interpretation, limitations, next steps, contributions

See the written sections in the milestone submission.

Overall we have 2 models (Multinomial Logistic & Gradient Boosting(XGBoost)) that we have trained and tested for the prediction on going for it on 4th down. Based on the results the XGBoost did a better job at predicting more accurately to go for it on fourth down. Since we are using more recent years for the test dataset and trained on older data, we have identified with data and our intuition that coaches decision making has changed. Coaches are much more likely to go for it and our plots above will show this. Our predictions were showing more conservative playcalls vs the coaches actually deciding to go for it more. This can support the intuition that in today's NFL, coaches are more willing to break the old standard of conservative 4th down plays and gamble more to go for it.

Since we know that coaching decisions have changed more recently, we can retrain our model on more recent historical data to see if that can help better predict with today's measures. Also we can possibly identify another variable to bring back or take out in order to improve model predictions.

Contributions:

Chaikou - I built our fourth-down dataset from the NFL play-by-play data for 1999 to 2025. I filtered it to real fourth-down decisions and labeled each play as a go-for-it, a punt, or a field goal. I split the data so we trained on 1999–2021, tuned on 2022–2023 and saved 2024–2025 for the final test. I then built and tuned two models, a multinomial logistic regression and a gradient boosting model, and compared them to two simple baselines on the test seasons. Using the boosting model, I checked whether coaching behavior has changed over time. Coaches started going for it more often than the model expected around 2018, and the gap has kept growing since. The model also got less accurate in recent seasons, which backs that up. In 2024–25, the gap was biggest in certain combinations of field position and yards to go.

Dillon - I organized the code and supplemental information via github. I had completed the data preprocessing and loading into the R platform. Also did preliminary exploratory analysis to better understand the data. I was able to identify ways for us to use each season data and concatenate together. After Chaikou implemented model, I was able to write conclusion and interpret model.

AI Statement

We have used AI to assist in creating shells for the code structure and for giving us ideal steps to verify within our model structure. We would then iterate on our actual thoughts and implementation and make our adjustments as needed to best fit our model.

```
#install.packages("nfl4th")
```

```
library(nfl4th)
```